

Table 2: Open source observability tools

Tool	Purpose	Notable features
Prometheus	Metrics collection and alerting	CNCF project; great with Kubernetes
Grafana	Visualisation	Supports multiple data sources; rich dashboards
Loki	Log aggregation	Built by Grafana Labs; lightweight and scalable
Fluentd	Log collection and routing	Works well in cloud-native and container setups
Jaeger	Distributed tracing	Open source from Uber; scalable for microservices
Zipkin	Distributed tracing	Lightweight; simple setup
OpenTelemetry	Unified data collection (metrics, logs, traces)	Vendor-neutral, fast growing CNCF (Cloud Native Computing Foundation) project

tool supports multiple data sources beyond Prometheus. This makes it adaptable for displaying logs, metrics and traces.

Logging tools like Loki and Fluentd are frequently used by organisations. Loki operates as an efficient solution that integrates seamlessly with Grafana. The log data collection and routing capabilities of Fluentd make it an excellent choice for containerised environments.

The distributed tracing tools Jaeger (created by Uber) and Zipkin are widely used by developers. These tools help teams track request flows between services. This enables better identification of system bottlenecks and failures in complex systems.

OpenTelemetry is new and is being adopted rapidly because it provides a unified standard for metrics, logs and traces. Major technology companies are backing this project, helping it turn into a fundamental component of open source observability infrastructure.

Teams can achieve complete system visibility by combining these tools without the need for costly proprietary solutions (Table 2).

Observability in CI/CD pipelines

The present DevOps environment eliminates traditional software development cycles that span from coding to release over extended periods. Cloud platforms, together with agile practices, enable organisations to deploy updates multiple times throughout each day. The fast deployment of new code becomes possible through continuous integration and continuous deployment (CI/CD), which automate testing and release processes.

This fast pace of operations creates new security challenges. A minor bug that crept in during morning development could potentially reach production systems before the lunch break. Observability is an absolute necessity here because it serves as the essential protective mechanism. It provides much needed visibility to teams, which enables them to detect problems early and understand their source before

users experience any negative effects.

The CI/CD pipeline receives complete support from observability throughout its operational sequence. In the integration phase, logs and metrics from test environments reveal failing components or performance dips long before code reaches production. Real-time metrics and traces help track how the system behaves when a new version is tested on a small group of users, called a canary deployment. They also monitor the system during the full release. The system can stop or reverse a release immediately when error rates increase or response times become slow.

Once changes are live, observability shifts its focus to how real users are experiencing the system. Tracing tools enable users to track the complete journey of failing requests, which helps identify the faulty update service or configuration.

Open source solutions power much of this visibility. Prometheus tracks performance metrics, Grafana turns them into interactive dashboards and Jaeger provides deep distributed tracing. These tools enable teams to maintain quick development speeds while maintaining complete control of their operations.

Observability in high velocity delivery environments creates trust by providing more than disaster prevention capabilities (Table 3).

Table 3: Observability in CI/CD pipelines

Pipeline phase	Role of observability	Benefits
Integration	Logs and metrics reveal failing tests	Early detection of issues
Deployment	Real-time metrics and traces monitor rollout	Immediate roll-back on failure
Production	Tracing tracks user request flows	Pinpoints faulty updates or configs